

Diagramas de flujo

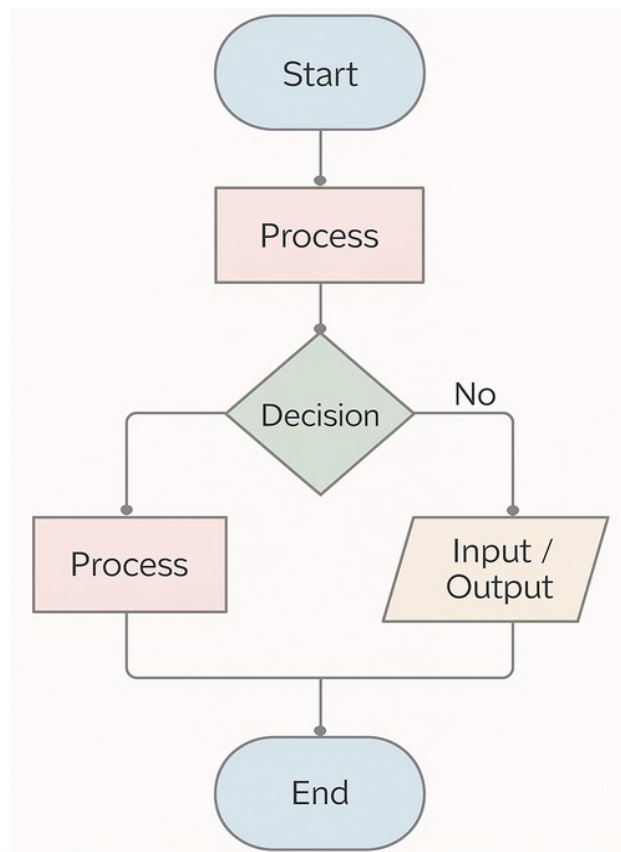
Índice

¿Qué es un Diagrama de Flujo?	3
Símbolos Comunes en los Diagramas de Flujo	4
Pasos para crear Diagramas de Flujo	4
Importancia de los diagramas de flujo en programación	5
Uso de pseudocódigo	5
Ejemplos	5
Ejemplo 1: Diagrama de flujo para sumar dos números	5
Ejemplo 2: Calcular el Promedio de Tres Números	6
Ejemplo 3: Determinar si un Número es Par o Impar	7
Ejemplo 4: Bucle para Sumar Números hasta un Límite	9

Los diagramas de flujo ("**flowcharts**" en inglés), son herramientas gráficas que representan visualmente el flujo de ejecución de un programa o proceso, es decir, son representaciones gráficas que muestran la lógica de un algoritmo o programa.

En programación se utilizan para planificar, diseñar y depurar algoritmos, facilitando la comprensión de la lógica, antes de escribir código. Este tutorial explica qué son los diagramas de flujo, sus componentes, su importancia en programación, y proporciona ejemplos prácticos en diferentes lenguajes de programación.

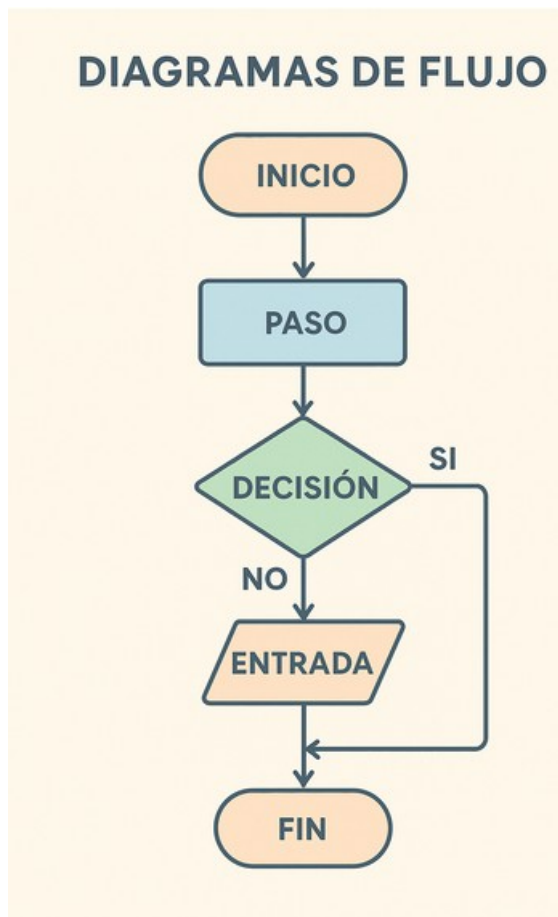
¿Qué es un Diagrama de Flujo?



Un diagrama de flujo es una representación gráfica de un proceso o algoritmo, que utiliza símbolos estandarizados para mostrar pasos, decisiones y flujos de control. En programación, los diagramas de flujo ayudan a:

- **Planificar algoritmos:** Visualizar la lógica antes de codificar.
- **Entender y planificar la lógica de un programa:** antes de codificar.
- **Comunicar ideas:** Explicar procesos complejos de manera clara a otros desarrolladores o interesados.
- **Depurar errores:** Identificar problemas en la lógica del programa.
- **Documentar procesos:** Servir como documentación para futuros mantenimientos.

Símbolos Comunes en los Diagramas de Flujo



Los diagramas de flujo utilizan símbolos estandarizados, cada uno con un propósito específico:

- **Óvalo:** Representa el inicio o fin de un proceso.
- **Rectángulo:** Indica un paso o instrucción (proceso).
- **Rombo:** Representa una decisión (condición lógica, como un "si-entonces").
- **Flechas:** Indican el flujo o dirección del proceso.
- **Paralelogramo:** Representa entrada o salida de datos (por ejemplo, leer o mostrar información).
- **Círculo pequeño:** Conector que une partes del diagrama, útil en diagramas grandes.

Existen muchos más símbolos como veremos al usar un editor visual de Diagramas de Flujo.

Pasos para crear Diagramas de Flujo

Veamos los pasos para crear un buen Diagrama de Flujo. Si te das cuenta, hasta casi el final no empiezas a dibujar el Diagrama. Los primeros pasos son para entender claramente lo que queremos hacer.

1. **Identificar el objetivo:** Define claramente qué problema resuelve el programa.
2. **Descomponer el proceso:** Divide el problema en pasos lógicos.
3. **Seleccionar los símbolos adecuados:** Usa los símbolos correspondientes para cada tipo de acción.
4. **Dibujar el flujo:** Conecta los pasos con flechas para mostrar la secuencia.

5. **Validar el diagrama:** Asegúrate de que el flujo cubre todos los casos posibles, incluyendo excepciones.

Importancia de los diagramas de flujo en programación

Nuestro cerebro funciona de manera muy visual, por eso es más sencillo compartir un diagrama de flujo que un texto con una explicación detallada.

- **Claridad:** Simplifican la comprensión de algoritmos complejos.
- **Colaboración:** Facilitan la comunicación entre equipos de desarrollo.
- **Eficiencia:** Ayudan a identificar redundancias o errores antes de codificar.
- **Enseñanza:** Son útiles para enseñar conceptos de programación a principiantes.

Uso de pseudocódigo

El pseudocódigo es una herramienta clave en el desarrollo de software, ya que actúa como un puente entre el diagrama de flujo y el código final en Python. Mientras que el diagrama de flujo visualiza la lógica y el flujo del programa de manera gráfica, el pseudocódigo traduce esa lógica en un formato textual, más cercano al lenguaje humano, pero estructurado.

Este paso nos va permitir planificar y diseñar la solución de forma clara, identificar errores lógicos y optimizar procesos antes de escribir el código en Python, lo que reduce errores, ahorra tiempo y facilita la colaboración entre programadores al ser independiente de un lenguaje específico.

Ejemplos

Vamos a ver algunos ejemplos, como la intención es ver todo el proceso, vamos a incluir sencillos programas en Python, que se comprenden con facilidad.

No te preocupes si no conoces la sintaxis todavía, en breve la aprenderemos. Afortunadamente Python genera una estructura muy visual en los programas para entender cómo se agrupan las sentencias.

También vamos a introducir el concepto de prueba o **test** para validar nuestra solución.

Ejemplo 1: Diagrama de flujo para sumar dos números

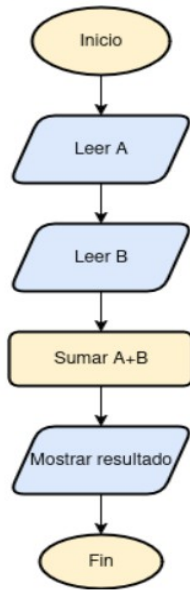
Descripción:

Leer dos números, sumarlos y mostrar el resultado:

Pseudocódigo:

```
Inicio
  Leer A
  Leer B
  Sumar A + B → Resultado
  Mostrar Resultado
Fin
```

Diagrama de flujo:



1. Inicio (Óvalo).

1. **Inicio** (Óvalo).

2. **Entrada:** Leer A (Paralelogramo).

3. **Entrada:** Leer B (Paralelogramo).

4. **Proceso:** Sumar los dos números (Rectángulo).

5. **Salida:** Mostrar el resultado (Paralelogramo).

6. **Fin** (Óvalo).

Código en Python

```
a = float(input("Ingrese A: "))
b = float(input("Ingrese B: "))
suma = a + b
print(f"La suma es: {suma}")
```

Ejemplo 2: Calcular el Promedio de Tres Números

Descripción

Este diagrama de flujo calcula el promedio de tres números ingresados por el usuario.

Es una sencilla variante, pero lo incluimos para ver que se puede simplificar un poco el diagrama y no hace falta incluir de forma repetitiva algunos pasos. Más adelante lo haremos con bucles....

Diagrama de flujo

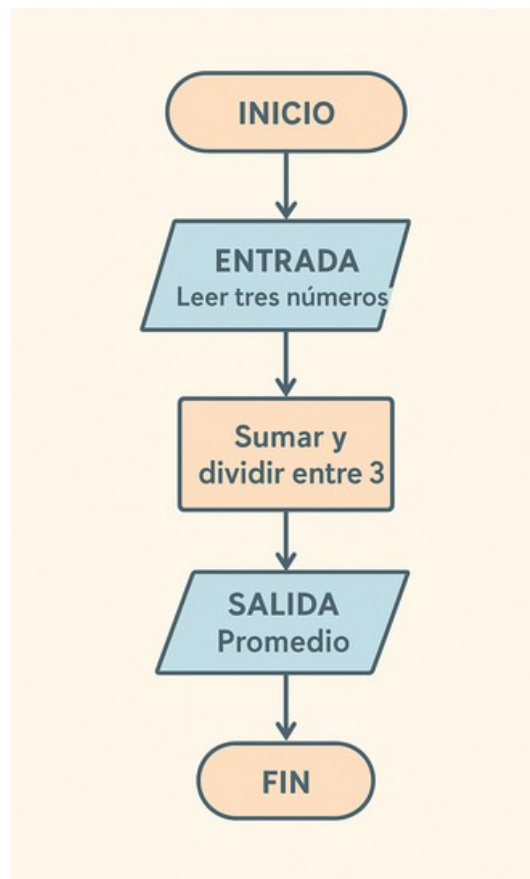
1. **Inicio** (Óvalo).

2. **Entrada:** Leer tres números (Paralelogramo).

3. **Proceso:** Sumar los números y dividir entre 3 (Rectángulo).

4. **Salida:** Mostrar el promedio (Paralelogramo).

5. **Fin** (Óvalo).



Código en Python

```
num1 = float(input("Ingrese el primer número: "))
num2 = float(input("Ingrese el segundo número: "))
num3 = float(input("Ingrese el tercer número: "))
promedio = (num1 + num2 + num3) / 3
print(f"El promedio es: {promedio}")
```

Explicación

- El programa solicita tres números al usuario.
- Calcula el promedio dividiendo la suma por 3.
- Muestra el resultado.

Probar y depurar

- Ejecuta el programa con diferentes valores (por ejemplo, 10, 20, 30).
- Verifica que el promedio sea correcto ($60 / 3 = 20$).
- Asegúrate de manejar posibles errores, como entradas no numéricas (puedes agregar un bloque try-except).

Ejemplo 3: Determinar si un Número es Par o Impar

Descripción

Este diagrama de flujo verifica si un número ingresado es par o impar.

Pseudocódigo:

```
Inicio
  Leer Número
  Si Número mod 2 = 0 entonces
```

```

    Mostrar "Par"
Sino
    Mostrar "Impar"
Fin

```

Diagrama de flujo

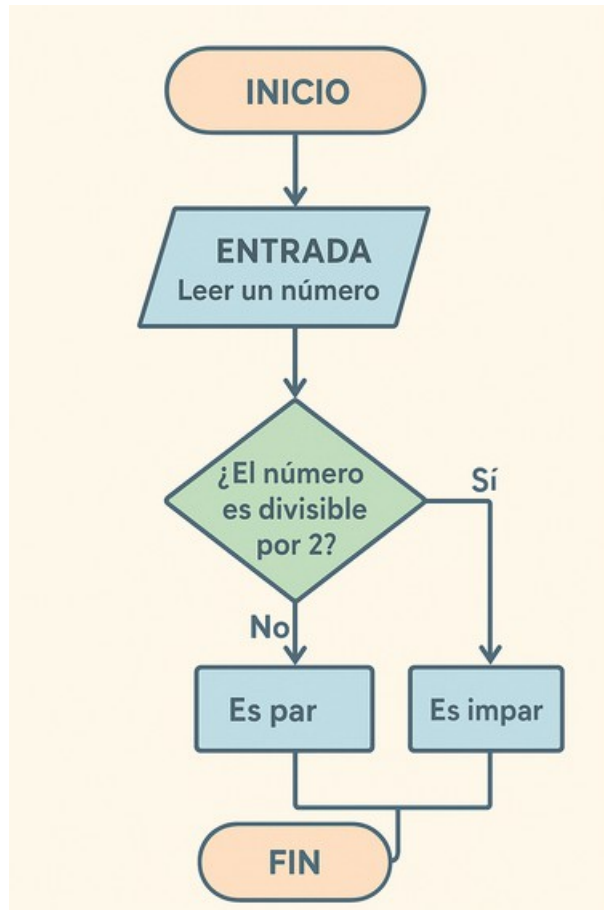


Diagrama de flujo

1. **Inicio** (Óvalo).
2. **Entrada:** Leer un número (Paralelogramo).
3. **Decisión:** ¿El número es divisible por 2? (Rombo).
4. Sí: Mostrar "Es par" (Paralelogramo).
5. No: Mostrar "Es impar" (Paralelogramo).
6. **Fin** (Óvalo).

Código Python

```

numero = input("Introduce un número: ")
if numero % 2 == 0:
    print("Es par")
else:
    print("Es impar")

```

Explicación

- El programa pide un número al usuario.

- Usa el operador módulo (%) para verificar si el número es divisible por 2.
- Muestra si es par o impar según el resultado.

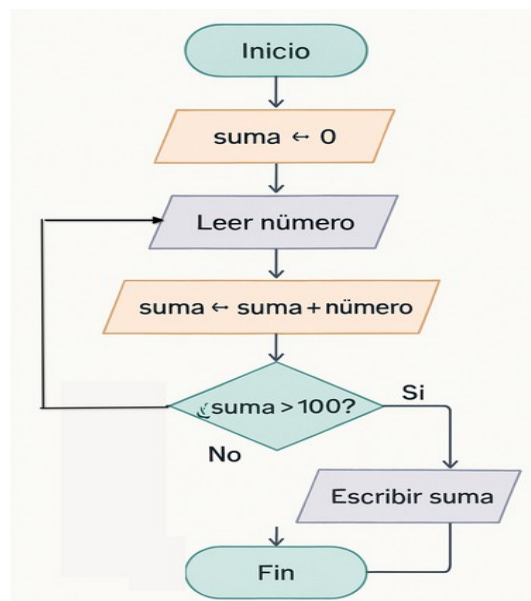
Probar y depurar

- Prueba con números como 4 (par) y 7 (impar).
- Asegúrate de manejar errores de entrada (por ejemplo, si el usuario ingresa texto en lugar de un número).

Ejemplo 4: Bucle para Sumar Números hasta un Límite

Descripción

Este diagrama de flujo suma números ingresados por el usuario hasta que la suma supere 100.



1. **Inicio** (Óvalo).
2. **Inicialización**: Suma = 0 (Rectángulo).
3. **Entrada**: Leer un número (Paralelogramo).
4. **Proceso**: Sumar el número a la variable suma (Rectángulo).
5. **Decisión**: ¿Suma > 100? (Rombo). - Sí: Mostrar suma y terminar (Paralelogramo, Óvalo). - No: Volver a leer un número (Regresa al paso 3).
6. **Fin** (Óvalo).

Código Python

```

suma = 0
while suma <= 100:
    numero = input("Introduzca un número: ")
    suma += numero

print("La suma total es: ", suma)

```

Explicación

- Inicializa la variable `suma` en 0.
- En un bucle, pide números al usuario y los suma.
- Cuando la suma supera 100, muestra el resultado y termina.